LAB 12

Lab Title: Pointers in C++

1. OBJECTIVE

The purpose of this lab is to understand computer memory organisation and to learn how pointers work in C++, including pointer declaration, the referencing operator (&), the dereferencing operator (*), and pointer arithmetic used to traverse arrays.

2. THEORY

Computer memory is made up of bytes, each identified by a unique address. When a variable is declared, the compiler reserves a specific block of memory to hold its value. The size of that block depends on the data type — for example, an integer typically occupies 4 bytes on a 32-bit system.

A pointer is a variable that stores the memory address of another variable rather than a data value directly. Key concepts include:

- Pointer Declaration — declared using an asterisk (*) before the variable name along with the data type it will point to. Example: int *ptr;
- Referencing Operator (&) — retrieves the memory address of a variable. Example: ptr = &k; makes ptr point to k.
- Dereferencing Operator (*) — accesses the value stored at the address held by the pointer. Example: *ptr = 7; sets the value of k to 7.
- Pointer Arithmetic — a pointer can be incremented or decremented to move through contiguous memory locations, especially useful when traversing arrays.

3. SOFTWARE USED

- DEV-C++ IDE
- C++ Compiler (bundled with DEV-C++)

4. PROCEDURE

Task 1 — Pointer Declaration and Initialization

Declare an integer variable called num and initialize it with a value of your choice. Declare a pointer variable called ptr of type integer. Initialize the pointer to point to the address of num. Display the value of num and the value pointed to by ptr using the dereference operator.

```
#include <iostream>
using namespace std;

int main()
{
    int num = 42;              // declare and initialize integer
    int *ptr;              // declare pointer to int
    ptr = &num;              // store address of num in ptr
    cout << "Value of num  : " << num  << endl;  // direct access
    cout << "Value via ptr : " << *ptr << endl;  // dereferenced access
    cout << "Address of num: " << &num << endl;  // memory address
    return 0;
}
```

Output:

```
Value of num  : 42
Value via ptr : 42
Address of num: 0x61ff08
```

Task 2 — Pointer Arithmetic

Declare an integer array called numbers with some values of your choice. Declare a pointer variable called ptr of type integer and initialize it to point to the first element of the array. Use pointer arithmetic to access and display each element of the array using the pointer.

```
#include <iostream>
using namespace std;

int main()
```

```
{
    int numbers[] = {10, 20, 30, 40, 50};  // declare array
    int *ptr = numbers;              // pointer to first element

    for (int i = 0; i < 5; i++)       // traverse using pointer
arithmetic
    {
        cout << "Element " << i << " : " << *ptr << endl;
        ptr++;                      // advance to next element
    }
    return 0;
}
```

Output:

```
Element 0 : 10
Element 1 : 20
Element 2 : 30
Element 3 : 40
Element 4 : 50
```

5. CONCLUSION

This lab introduced the concept of pointers in C++ and their relationship to computer memory. A pointer variable was declared and used to access and display the value of another variable through the dereference operator (*). Pointer arithmetic was then applied to traverse the elements of an integer array sequentially, demonstrating how a pointer advances through contiguous memory locations. The distinction between the address a pointer stores and the value at that address was clearly understood.